

Salvemos a las nutrias

Una reserva de nutrias quiere buscar soluciones a algunas cuestiones que ocurren en sus instalaciones

Las nutrias de la reserva envían mensajes sobre el estado del río, pero estos suelen contener ruido, inconsistencias y palabras irrelevantes. Se pide crear una clase estática **ProcesadorAlertas** dentro del paquete correspondiente(reserva.mensajes), que permita procesar estos mensajes.

La clase debe incluir **al menos los siguientes métodos**, cada uno encargado de una fase del procesamiento:

1. Método de normalización del mensaje

Recibe un mensaje original en forma de texto.

Debe:

- eliminar espacios sobrantes al inicio y al final,
- sustituir secuencias de varios espacios por uno solo,
- convertir todo el texto a minúsculas,
- eliminar cualquier carácter que no sea letra o espacio,
- devolver el mensaje ya normalizado.

2. Método para dividir el mensaje en palabras

Recibe un **mensaje normalizado**.

Debe:

- dividir el mensaje en palabras individuales,
- ignorar fragmentos vacíos,
- construir y devolver un array con todas las palabras sin huecos ni posiciones sobrantes.

3. Método para contar palabras clave

Recibe dos arrays:

- las palabras extraídas del mensaje,
- un conjunto de palabras clave definido externamente.

Debe:

- comparar las palabras del mensaje con las palabras clave,
- contar cuántas coinciden,
- devolver la cantidad de coincidencias.

4. Programa principal con menú

El programa principal debe implementar un menú interactivo con las siguientes opciones:

El menú **debe permitir**:

- introducir o seleccionar un mensaje,
- mostrar el mensaje normalizado,
- mostrar las palabras del mensaje,
- contar cuántas palabras clave contiene,
- ejecutar el proceso completo en una sola opción,
- salir del programa.

Habrá que filtrar que si se intenta utilizar un método que requiere algún paso anterior y si eso no existe todavía se le avise al usuario.

EJERCICIO 2 — Evaluación de seguridad en recorridos del río (versión definitiva con matrices)

Las nutrias deben cruzar distintos tramos del río. Cada tramo se denomina **zona**, y cada zona tiene tres valores:

1. nivel del agua,
2. velocidad de la corriente,
3. número de obstáculos.

Se pide crear una clase estática **ControlZonas**, dentro del paquete indicado(reserva.zonas), que permita analizar estas zonas y también recorridos completos.

1. Método para determinar si una zona es segura

Debe implementarse un método que reciba **tres valores enteros**:

- el nivel de agua,
- la velocidad de la corriente,
- el número de obstáculos.

Este método debe determinar si la zona es segura aplicando todas estas reglas:

- el nivel del agua debe estar entre **30 y 65**,
- la velocidad de la corriente debe ser **menor que 45**,
- debe haber **menos de 3 obstáculos**,
- si la velocidad de la corriente es **30 o más**, la zona solo puede considerarse segura cuando **el número de obstáculos es 0**.

El método debe devolver `true` si la zona cumple todas las reglas anteriores, y `false` si incumple alguna.

2. Método para evaluar un recorrido completo

Un recorrido del río se representará mediante una **matriz bidimensional**, donde:

- cada **fila** representa una zona,
- cada **columna** representa un dato de esa zona:

columna 0 → nivel del agua

columna 1 → velocidad de la corriente

columna 2 → número de obstáculos

Cada recorrido puede tener un número indeterminado de zonas.

Por ejemplo, una matriz con 4 zonas podría ser:

```
{  
  {40, 20, 1},  
  {55, 31, 0},  
  {33, 18, 2},  
  {29, 15, 0}  
}
```

Se debe implementar un método que reciba la matriz de un recorrido.

Este método debe:

- recorrer todas las filas de la matriz,

- obtener los tres valores de cada zona,
- evaluar cada zona usando el método definido en el apartado 1,
- contar cuántas zonas son seguras,
- devolver ese número total.

3. Programa principal con menú

El método principal debe implementar un menú interactivo.

El programador deberá **definir varios recorridos** del río en forma de matrices bidimensionales directamente en el main (por ejemplo, Recorrido A, Recorrido B, Recorrido C).

El menú **debe permitir al usuario**:

- seleccionar uno de los recorridos disponibles,
- visualizar los datos de todas las zonas del recorrido seleccionado,
- elegir una zona concreta e indicar si es segura aplicando el método descrito en el punto 1,
- evaluar el recorrido completo aplicando el método del punto 2,
- cambiar de recorrido,
- salir del programa.

EJERCICIO 3 — Modelo de nutridas, amistades y colonias (POO, visibilidad, composición y campos estáticos)

La reserva quiere modelar colonias de nutridas para estudiar su comportamiento social. Se pide implementar las siguientes clases en los paquetes indicados:

`reserva.modelo`

`Amistad`

`Nutrida`

`Colonia`

`reserva.util`

`GestorColonias`

Todas las clases deben respetar las visibilidades, usar arrays cuando se indique, y emplear composición sin herencia ni excepciones.

1. Clase Amistad

Amistad

**Atributos
privados**

`otra : Nutrida`

`intensidad : int`

Campo estático

`INTENSIDAD_MINIMA_ALTA : int = 7`

Constructor	Recibe <code>otra : Nutrida</code> e <code>intensidad : int</code> . Debe validar la intensidad usando el método privado.
Métodos públicos	<code>getOtra() : Nutrida</code> <code>getIntensidad() : int</code> <code>esIntensa() : boolean</code> (true si <code>intensidad >= INTENSIDAD_MINIMA_ALTA</code>)
Método privado	<code>validarIntensidad(valor:int) : boolean</code> → Devuelve true si $1 \leq \text{valor} \leq 10$.
Instrucción obligatoria	Si <code>validarIntensidad(intensidad)</code> devuelve false, el constructor debe asignar <code>intensidad = 1</code> .

2. Clase Nutrida

Nutrida

Atributos privados	<code>nombre : String</code> <code>edad : int</code> <code>peso : double</code> <code>amistades : Amistad[]</code> (hasta 5 posiciones)
Campo estático	<code>totalNutridasCreadas : int</code> (se incrementa en cada constructor)

Constructores	1) nombre, edad, peso → crea nutrida sin amistades. 2) nombre, edad, peso, amistades → copia real del array recibido.
Métodos públicos	<pre> getNombre() : String getEdad() : int getPeso() : double getAmistades() : Amistad[] esJuvenil() : boolean (edad < 3) contarAmistadesIntensas() : int tieneAmistadCon(otra:Nutrida) : boolean </pre>
Método privado	<pre> contarAmistadesReales() : int (cuenta cuántas posiciones del array amistades no son null) </pre>
Instrucción obligatoria	Cuando se reciba un array en el constructor, no debe asignarse directamente: <pre>✗ this.amistades = amistades;</pre> Debe copiarse elemento a elemento creando un array nuevo: <pre>✓ this.amistades = new Amistad[amistades.length];</pre> ✓ copiar cada elemento en un bucle: <pre> for (int i = 0; i < amistades.length; i++) { this.amistades[i] = amistades[i]; } </pre>

3. Clase Colonia

Colonia

Atributos privados

nombre : String
integrantes : Nutrida[]

Campos estáticos

CAPACIDAD_MAXIMA : int = 20
coloniasCreadas : int

Constructores

1) nombre + array nutridas
2) nombre → colonia vacía (new Nutrida[0])

Métodos públicos

getNombre() : String
getIntegrantes() : Nutrida[]
pesoMedio() : double
buscarPorNombre(nombre:String) : Nutrida
contarEnlacesAmistad() : int

Método privado

contarNutridasReales() : int → recorre el array integrantes y **cuenta cuántas posiciones no son null.**

Uso del método privado

Otros métodos (como pesoMedio o contarEnlacesAmistad) pueden apoyarse en él para no recorrer posiciones vacías.

4. Clase GestorColonias

(todos los métodos deben ser estáticos)

GestorColonias

Método 1 `crearColoniaFiltrada(origen:Colonia, pesoMinimo:double) : Colonia`

— Debe construir una nueva colonia con nutridas cuyo peso sea $\geq$ pesoMinimo.

Método 2 `sonCompatibles(a:Nutrida, b:Nutrida) : boolean`

— Devuelve true si:

- la diferencia de edad es ≤ 5 años,
- el peso conjunto no supera 25,
- ambas nutridas tienen menos de 5 amistades reales.

5. Programa principal (main)

El programa principal debe demostrar el funcionamiento del modelo completo realizando al menos lo siguiente:

1. Crear varias nutridas usando ambos constructores.
2. Crear varias amistades y asociarlas correctamente a nutridas.
3. Mostrar si cada amistad es intensa o no.
4. Crear al menos dos colonias (una con array inicial y otra vacía).
5. Usar métodos de Nutrida:
 - `esJuvenil()`,

- `tieneAmistadCon(...)`,
- `contarAmistadesIntensas()`.

6. Usar métodos de Colonia:

- `pesoMedio()`,
- `buscarPorNombre(...)`,
- `contarEnlacesAmistad()`.

7. Usar los métodos de GestorColonias:

- `crearColoniaFiltrada(...)`,
- `sonCompatibles(...)`.

El programa debe mostrar los resultados por consola.